المقدمة لنظم التشغيل - المفاهيم الثمانية الأساسية

🎯 مقدمة: من مجرد برنامج إلى نظام حي

كيف يعمل الحاسوب فعلياً؟

عندما تفتح برنامج على الحاسوب، تحدث عمليات معقدة خلف الكواليس . نظام التشغيل يتحول من مجرد كود إلى "مدير" ذكي يدير الموارد ويحمي النظام.

المرحلة 1: البرنامج الساكن (Program)

- ملف على القرص الصلب
- لا يستهلك CPU أو Memory
- مثل: ملف Word.exe قبل فتحه

المرحلة 2: العملية الحية (Process) ⭐

- البرنامج يصبح قيد التنفيذ
- يحتاج موارد: CPU، Memory، I/O
- مثل: برنامج Word بعد النقر المزدوج عليه

التحول:

من "كيف أشغل البرنامج؟" إلى "كيف يدير نظام التشغيل مئات العمليات معاً؟"

التطبيقات الواقعية

1. **عند فتح متصفح Chrome:**
   - يتحول من Program إلى Process
   - يحتاج CPU للمعالجة
   - يحتاج Memory لحفظ البيانات
2. **عند تحميل ملف:**
   - القرص الصلب يرسل Interrupt
   - CPU توقف عملها الحالي
   - تنفذ Interrupt Handler لمعالجة التحميل
3. **عند الطباعة:**
   - البرنامج يستدعي System Call
   - ينتقل من User Mode إلى Kernel Mode
   - نظام التشغيل يدير الطابعة

🔔 القسم الأول: Interrupt - المقاطعة

التعريف الأساسي

Interrupt (المقاطعة):

إشارة من Hardware أو Software توقف CPU عن عمله الحالي لتنفيذ Interrupt Handler (معالج المقاطعة)

الآلية:

- CPU تعمل على Process A
- يحدث Interrupt (قرص صلب أنهى قراءة)
- CPU توقف A وتحفظ حالته
- CPU تنفذ Interrupt Handler
- CPU ترجع لـ A وتكمل عمله

الأنواع:

- **Hardware Interrupt:** من الأجهزة (لوحة مفاتيح، قرص صلب، طابعة)
- **Software Interrupt:** من البرامج (System Call، Exception)

المصطلحات الأساسية

1. Interrupt Handler (معالج المقاطعة):

روتين صغير في نظام التشغيل يتعامل مع سبب المقاطعة

مثال: قرص صلب أنهى قراءة ملف 500KB → Interrupt Handler يحدث Buffer ويخبر البرنامج

2. Interrupt Vector (جدول المقاطعات):

جدول يحتوي على عناوين كل Interrupt Handlers

مثال:

- Interrupt #14 = عنوان معالج القرص الصلب
- Interrupt #1 = عنوان معالج لوحة المفاتيح

3. Interrupt Service Routine (ISR):

اسم آخر لـ Interrupt Handler

أنواع Interrupts

النوع	المصدر	أمثلة	قابل للتجاهل؟
Hardware Interrupt	الأجهزة الخارجية	لوحة المفاتيح، الفأرة، القرص الصلب، الطابعة	Maskable (معظمها)
Timer Interrupt	ساعة النظام	Time Slice انتهى	Non-maskable
I/O Interrupt	أجهزة الإدخال/الإخراج	قراءة/كتابة انتهت	Maskable
Software Interrupt	البرامج	System Call (TRAP)	Non-maskable
Exception	أخطاء البرنامج	Division by Zero، Page Fault	Non-maskable

CPU مثل موظف في دائرة حكومية:

الوضع العادي:

- يشتغل على معاملة مواطن (Process A)
- فجأة يدخل شخص مهم (Interrupt)
- يوقف المعاملة الحالية
- يحفظ الأوراق (Context Save)
- يخدم الشخص المهم (Interrupt Handler)
- يرجع للمعاملة الأولى (Context Restore)

⚠️ المشكلة الكبرى: كثرة المقاطعات

إذا كل شوية يدخل شخص مهم:

- الموظف ما يكمل أي معاملة
- الوقت يضيع في الحفظ والاسترجاع
- الإنتاجية تنخفض
- **النتيجة:** Overhead عالي

الحل:

- نظام التشغيل يجب يدير الـ Interrupts بذكاء
- بعض الـ Interrupts تكون **Non-maskable** (ما تنرفض - مثل Power Failure)
- بعضها **Maskable** (ممكن نأجلها - مثل لوحة المفاتيح)

خطوات معالجة Interrupt

1. حدوث Interrupt:

جهاز I/O يرسل إشارة للـ CPU

2. CPU تتحقق:

- هل Interrupts مفعلة؟
- ما هو نوع Interrupt؟

3. حفظ السياق:

- حفظ Program Counter
- حفظ Registers
- حفظ حالة CPU

4. تحديد المعالج:

- البحث في Interrupt Vector Table
- القفز لعنوان Interrupt Handler

5. تنفيذ المعالج:

- معالجة السبب (مثلاً: قراءة البيانات من Buffer)
- تحديث حالة العملية المنتظرة

6. استرجاع السياق:

- استرجاع Program Counter
- استرجاع Registers
- العودة للعملية المتوقفة

7. استكمال التنفيذ:

CPU تكمل من حيث توقفت

🔐 القسم الثاني: User Mode vs Kernel Mode

المشكلة الأساسية

مشكلة الحماية (Protection Problem):

معطى: برنامج يعمل على الحاسوب

المطلوب: كيف نمنعه من تخريب النظام أو سرقة البيانات؟

مثال:

برنامج فيروس يحاول:

- مسح ملفات النظام
- قراءة كلمات السر
- التحكم بالأجهزة مباشرة
- إيقاف نظام التشغيل

الحل:

- نعزل البرامج في **User Mode** = صلاحيات محدودة ✅
- نظام التشغيل في **Kernel Mode** = صلاحيات كاملة ✅

التعريف الأساسي

User Mode (وضع المستخدم):

- البرامج العادية تعمل هنا
- وصول محدود للموارد
- لا تستطيع تشغيل Privileged Instructions (تعليمات خطيرة)
- لا تستطيع الوصول المباشر للـ Hardware

Kernel Mode (وضع النواة):

- نظام التشغيل يعمل هنا
- وصول كامل للـ Hardware
- تستطيع تشغيل أي تعليمة
- تستطيع الوصول لكل الذاكرة

المقارنة الشاملة

الميزة	User Mode	Kernel Mode
الصلاحيات	محدودة	كاملة
الوصول للـ Hardware	ممنوع	مسموح
Privileged Instructions	❌	✅
الوصول للذاكرة	ذاكرة البرنامج فقط	كل الذاكرة
من يعمل هنا	البرامج العادية	نظام التشغيل
السرعة	عادية	عادية
الأمان	محمي (لا يؤثر على النظام)	خطر (خطأ = crash)

المفهوم الذهبي: Mode Switch ⭐

تبديل الوضع (Mode Switch):

"كيف ينتقل البرنامج من User Mode إلى Kernel Mode؟"

الطريقة الوحيدة: System Call

الآلية:

1. البرنامج يحتاج خدمة (فتح ملف)
2. يستدعي System Call مثل `open()`
3. تعليمة **TRAP** تنفذ
4. CPU تنتقل تلقائياً إلى Kernel Mode
5. نظام التشغيل ينفذ الخدمة
6. CPU ترجع إلى User Mode

⚠️ **تحذير مهم:**

إذا حاول البرنامج الانتقال مباشرة إلى Kernel Mode:

- **Exception** يحصل
- نظام التشغيل يوقف البرنامج فوراً
- رسالة خطأ: "Illegal Instruction" أو "Segmentation Fault"

التشبيه

مثل موظف في شركة:

- الموظف (User Mode) ما يقدر يفتح خزنة الشركة
- لازم يطلب من المدير (Kernel Mode)
- المدير يتأكد من الصلاحيات
- إذا مسموح، ينفذ الطلب ✅
- إذا ممنوع، يرفض ❌

Privileged Instructions (أمثلة)

تعليمات ممنوعة في User Mode:

- تعطيل/تفعيل Interrupts
- تعديل Interrupt Vector Table
- الوصول لأجهزة I/O مباشرة
- تعديل Memory Management registers
- تغيير Mode bit نفسه
- إيقاف CPU (HALT instruction)

تعليمات مسموحة في User Mode:

- العمليات الحسابية (ADD, SUB, MUL, DIV)
- القراءة/الكتابة في ذاكرة البرنامج
- القفز داخل البرنامج (JUMP, BRANCH)
- استدعاء System Call (TRAP)

خطوات System Call بالتفصيل

الخطوة 1: الإعداد (في User Mode)

- البرنامج يضع رقم System Call في Register معين
- يضع المعاملات (Parameters) في Registers أو Stack
- مثال: `open("file.txt", O_RDONLY)`

الخطوة 2: تنفيذ TRAP

- تعليمة TRAP (Software Interrupt) تنفذ
- CPU تحفظ السياق تلقائياً:
  - Program Counter (عنوان التعليمة التالية)
  - Registers الحالية
  - Processor Status

الخطوة 3: Mode Switch

- Mode Bit يتغير: 1 (User) → 0 (Kernel)
- CPU الآن في Kernel Mode ✅

الخطوة 4: القفز للمعالج

- CPU تقرأ System Call Number
- تبحث في System Call Table
- تقفز لعنوان المعالج المناسب

الخطوة 5: التنفيذ (في Kernel Mode)

- نظام التشغيل ينفذ الخدمة:
  - يتحقق من الصلاحيات
  - ينفذ العملية المطلوبة
  - يضع النتيجة في Register

الخطوة 6: العودة

- استرجاع السياق المحفوظ
- Mode Bit يتغير: 0 (Kernel) → 1 (User)
- CPU ترجع للبرنامج

الخطوة 7: الاستمرار (في User Mode)

- البرنامج يكمل من حيث توقف
- يقرأ النتيجة من Register

مثال تفصيلي: فتح ملف

السيناريو:

برنامج يريد فتح ملف "data.txt"

المرحلة	Mode	PC	الإجراء
1	User	1000	البرنامج: `fd = open("data.txt", O_RDONLY)`
2	User	1004	تعليمة TRAP تحضر للتنفيذ
3	**Switch**	1004	حفظ PC=1004, Mode → Kernel
4	Kernel	50000	القفز لـ System Call Handler
5	Kernel	50000	التحقق: هل الملف موجود؟ ✅
6	Kernel	50010	التحقق: هل المستخدم له صلاحية؟ ✅
7	Kernel	50020	فتح الملف، إنشاء File Descriptor = 3
8	Kernel	50030	وضع النتيجة (3) في Register EAX
9	**Switch**	1004	استرجاع PC=1004, Mode → User
10	User	1004	البرنامج يكمل: fd الآن = 3 ✅

الحماية:

- البرنامج ما قدر يفتح الملف مباشرة ✅
- نظام التشغيل تحقق من الصلاحيات ✅
- لو كان الملف محمي، System Call ترجع -1 ❌

🔄 القسم الثالث: Process - العملية

التعريف الأساسي

Process (العملية):

برنامج قيد التنفيذ (Program in Execution)

المكونات الأساسية:

- **Code (Text Segment):** الكود البرمجي (read-only)
- **Data:** المتغيرات العامة المهيأة
- **BSS:** المتغيرات العامة غير المهيأة
- **Heap:** الذاكرة الديناميكية (malloc, new)
- **Stack:** المكدس للدوال والمتغيرات المحلية

Process vs Program

الميزة	Program	Process
التعريف	ملف تنفيذي على القرص	برنامج قيد التنفيذ
الحالة	ساكن (Passive)	نشط (Active)
الذاكرة	لا يستهلك RAM	يستهلك RAM
CPU	لا يستخدم CPU	يستخدم CPU
العدد	واحد	يمكن عدة نسخ
مثال	Word.exe على القرص	نافذتان من Word مفتوحتان

مثال واقعي:

- **Program:** ملف Word.exe حجمه 50MB على `\C:\Program Files`
- **Process A:** Word يفتح ملف report.docx (يستهلك 200MB RAM)
- **Process B:** Word يفتح ملف cv.docx (يستهلك 180MB RAM منفصلة)

Process Control Block (PCB)

PCB (Process Control Block):

هيكل بيانات يحفظ كل معلومات العملية

المكونات الأساسية:

1. معلومات التعريف:

- Process ID (PID) - رقم معرف فريد
- Parent Process ID (PPID) - رقم العملية الأب
- User ID - صاحب العملية

2. حالة CPU:

- Program Counter (PC) - عنوان التعليمة التالية
- Registers - نسخة من السجلات (32 register عادة)
- Stack Pointer - مؤشر المكدس
- Processor Status Word (PSW)

3. معلومات الجدولة:

- Process State - الحالة الحالية (New/Ready/Running/Waiting/Terminated)
- Priority - الأولوية
- CPU Time - الوقت المستهلك
- Arrival Time - وقت الوصول
- Waiting Time - وقت الانتظار

4. إدارة الذاكرة:

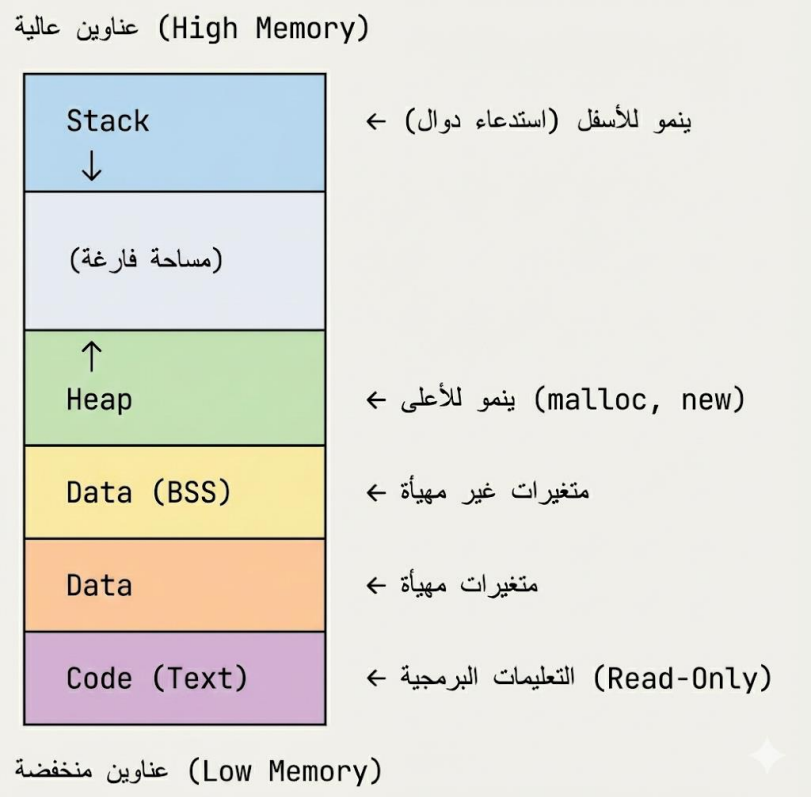
- Base Register - بداية الذاكرة
- Limit Register - حجم الذاكرة
- Page Table Pointer - مؤشر جدول الصفحات

5. معلومات I/O:

- Open Files - الملفات المفتوحة
- I/O Devices - الأجهزة المستخدمة
- Pending I/O Operations - عمليات I/O المنتظرة

Process في الذاكرة

تخطيط الذاكرة لـ Process:



مثال:

- `int globalVar = 10;` → في Data Segment
- `int uninitVar;` → في BSS Segment
- `void function() {...}` → Code في Text Segment
- `int localVar = 5;` → في Stack
- `int* ptr = new int;` → ptr في Stack، القيمة في Heap

🔄 القسم الرابع: Process States - حالات العملية

الحالات الخمس

1. New (جديدة):

- العملية قيد الإنشاء
- نظام التشغيل يخصص موارد لها
- يتم إنشاء PCB

2. Ready (جاهزة):

- جاهزة للتنفيذ تماماً
- تنتظر دورها للحصول على CPU
- موجودة في Ready Queue

3. Running (قيد التنفيذ):

- تعمل على CPU حالياً
- عدد الـ Running Processes = عدد CPUs
- في نظام Single-CPU: واحد فقط Running

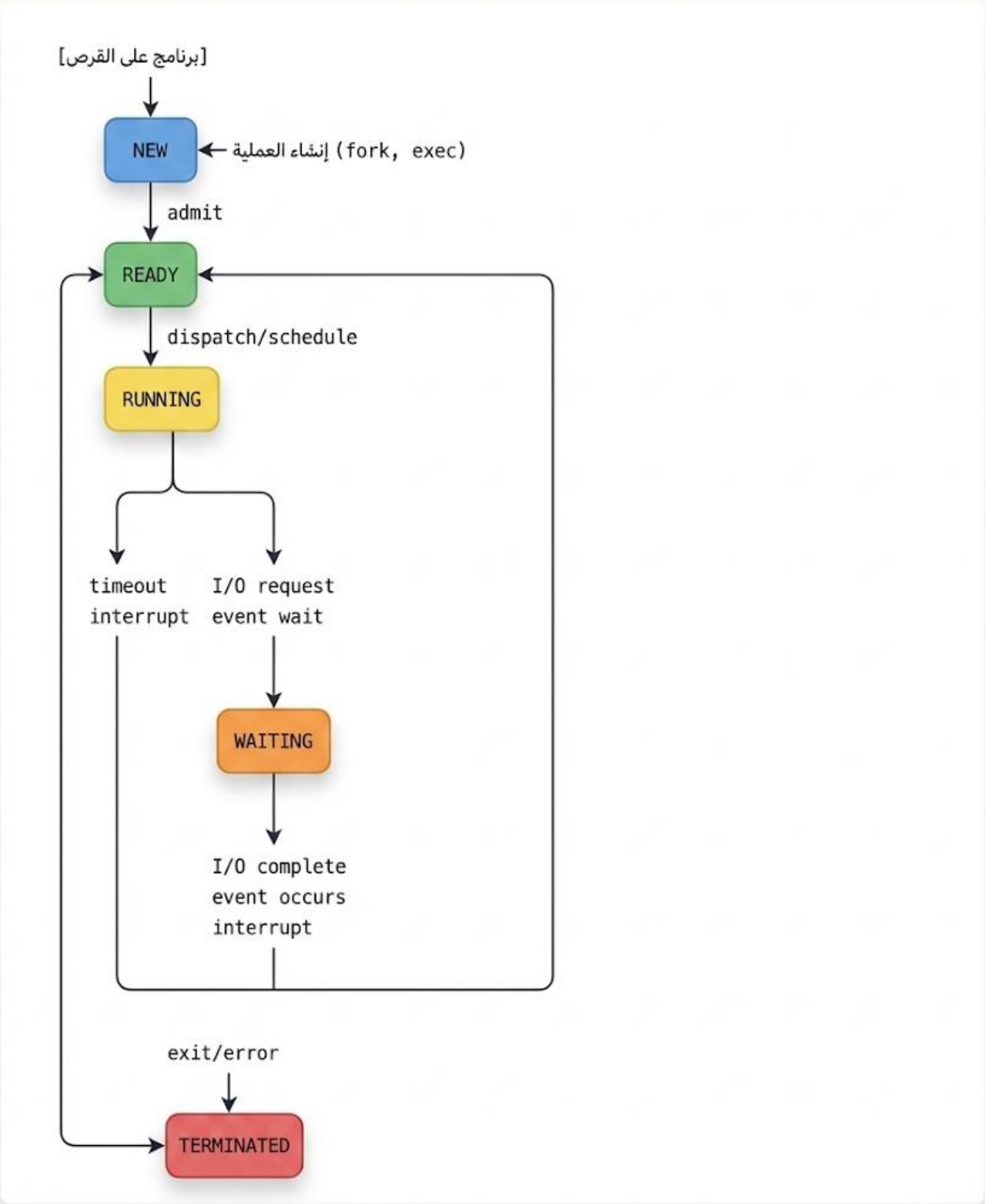
4. Waiting (منتظرة):

- تنتظر حدث معين
- مثل: انتظار قراءة ملف، انتظار إدخال مستخدم
- موجودة في Waiting Queue
- **لا تستهلك CPU** ⭐

5. Terminated (منتهية):

- انتهت العملية
- نظام التشغيل يحرر الموارد
- PCB يحذف

مخطط الحالات (State Diagram)



الانتقالات بين الحالات (Transitions)

الانتقال	الاسم	السبب	ملاحظات
New → Ready	Admit	نظام التشغيل جاهز لجدولة العملية	PCB جاهز
Ready → Running	Dispatch/Schedule	Scheduler اختار العملية	CPU خالي
Running → Ready	Timeout/Preemption	Time Quantum انتهى، أو عملية أعلى أولوية	Context Switch
Running → Waiting	I/O Request	طلب I/O (read, write, sleep)	لا تستهلك CPU
Waiting → Ready	I/O Complete	Interrupt: I/O انتهى	**ليس إلى Running مباشرة!** ⭐
Running → Terminated	Exit	البرنامج أنهى (exit) أو خطأ	تحرير موارد
Ready → Terminated	Kill	نظام التشغيل أنهى العملية قسراً	نادر
Waiting → Terminated	Kill	إنهاء أثناء الانتظار	نادر

⚠️ ملاحظات مهمة جداً للامتحان

1. Waiting → Ready (وليس Running!):

عندما I/O ينتهي:

- Process ينتقل من Waiting إلى **Ready** فقط
- يدخل Ready Queue
- ينتظر دوره في Scheduler
- **لا** ينتقل مباشرة إلى Running ❌

السبب: قد يكون CPU مشغول بـ Process آخر!

2. Running Process واحد فقط (في Single-CPU):

- في نظام به CPU واحد: Running Process = 1 فقط
- في نظام به 4 CPUs: Running Processes = 4 كحد أقصى

3. Ready vs Waiting:

- **Ready:** جاهز تماماً، ينتظر CPU فقط
- **Waiting:** ينتظر حدث خارجي (I/O)، **لا يستطيع** استخدام CPU حتى لو كان خالياً

📞 القسم الخامس: System Call - استدعاء النظام

التعريف الأساسي

System Call:

الطريقة الوحيدة التي يطلب بها البرنامج خدمة من نظام التشغيل

الخصائص:

- تسبب الانتقال من User Mode إلى Kernel Mode
- تستخدم تعليمة خاصة: TRAP أو INT (Software Interrupt)
- أبطأ من Function Call عادي (~15x)

تصنيف System Calls

1. Process Control:

- `fork()` - إنشاء عملية جديدة (نسخ العملية الحالية)
- `exec()` - تنفيذ برنامج جديد
- `wait()` - انتظار عملية ابن تنتهي
- `exit()` - إنهاء العملية
- `getpid()` - الحصول على Process ID

2. File Management:

- `open()` - فتح ملف
- `read()` - قراءة من ملف
- `write()` - كتابة إلى ملف
- `close()` - إغلاق ملف
- `lseek()` - تحريك مؤشر الملف

3. Device Management:

- `ioctl()` - التحكم بجهاز I/O
- `write`/`read()` - للأجهزة أيضاً (في Unix كل شيء ملف)

4. Information Maintenance:

- `getpid()` - رقم العملية
- `time()` - الوقت الحالي
- `sleep()` - إيقاف مؤقت
- `alarm()` - تعيين منبه

5. Communication:

- `pipe()` - قناة بين عمليتين (IPC)
- `socket()` - اتصال شبكي
- `recv`/`send()` - إرسال/استقبال بيانات

مثال تفصيلي: fork() System Call

fork() - إنشاء عملية جديدة:

التعريف:

`fork()` ينشئ نسخة طبق الأصل من العملية الحالية

القيمة المرجعة:

- **للـ Parent Process:** PID (رقم موجب) الابن
- **للـ Child Process:** 0
- **عند الفشل:** -1

التنفيذ خطوة بخطوة:

المرحلة	Parent Process	Child Process	x (Parent)	x (Child)
قبل fork	يعمل	غير موجود	10	-
عند fork	ينفذ System Call	يتم إنشاؤه	10	10
بعد fork	pid = 1234 (مثلاً)	pid = 0	10	10
Parent: if	pid != 0 → else	-	10	10
Child: if	-	pid == 0 → true	10	10
Parent: print	"Parent: x = 10"	-	10	10
Child: تعديل	-	x = 20	10	**20**
Child: print	-	"Child: x = 20"	10	20

النتيجة:

```text
Parent: x = 10
Child: x = 20
```

⭐ **نقاط مهمة للامتحان:**

1. **ذاكرة منفصلة:** Parent و Child لكل واحد ذاكرة مستقلة
2. **القيمة المرجعة مختلفة:** Parent يحصل على PID، Child يحصل على 0
3. **نفس Program Counter:** Child يبدأ من التعليمة بعد fork()
4. **نسخ PCB:** Child يأخذ نسخة من PCB الأب
5. **عمليتان مستقلتان:** تعملان بالتزامن (Concurrency)

System Call vs Function Call

الميزة	Function Call	System Call
Mode	نفس Mode (User عادة)	User → Kernel → User
السرعة	سريع (~15 cycles)	بطيء (~250 cycles)
الآلية	CALL instruction	TRAP instruction
Context Switch	لا	نعم (جزئي)
الحماية	لا توجد	نعم (Kernel يتحقق)
أمثلة	`strlen`, `sqrt()`	`fork`, `read()`, `open()`

لماذا System Calls بطيئة؟

التكلفة (Overhead):

العملية	الوقت (cycles)
Function Call عادي	~15
TRAP instruction	~20
Mode Switch (User→Kernel)	~50
Context Save	~30
Handler Execution	~100
Context Restore	~30
Mode Switch (Kernel→User)	~20
الإجمالي	**~250**

الحل:

- استخدم System Calls بحكمة
- اقرأ/اكتب Buffers كبيرة بدلاً من بايت واحد
- Library Functions (مثل `printf`) تجمع البيانات قبل System Call

🔀 القسم السادس: Context Switch - تبديل السياق

التعريف الأساسي

Context Switch (تبديل السياق):

حفظ حالة Process الحالي واستعادة حالة Process آخر

السياق (Context) يشمل:

- Program Counter (PC) - عنوان التعليمة التالية
- Registers - كل السجلات (32 register عادة)
- Stack Pointer - مؤشر المكدس
- Process State - الحالة
- Memory Management Info - معلومات الذاكرة

متى يحدث Context Switch؟

الأسباب الأربعة:

1. **Interrupt:**
   - Timer Interrupt (انتهى Time Slice)
   - I/O Interrupt (جهاز أنهى عمله)
2. **System Call:**
   - العملية طلبت `write`, `read()`: I/O`()`
   - العملية انتظرت: `sleep`, `wait()` `()`
3. **Time Slice Expiration:**
   - الوقت المخصص للعملية انتهى (عادة 10-100ms)
   - Scheduler يختار عملية أخرى
4. **Priority Change:**
   - وصول عملية ذات أولوية أعلى
   - Preemptive Scheduling

خطوات Context Switch

المراحل الخمس:

1. حفظ سياق Process القديم:

- `OldPCB.PC = CPU.programCounter`
- `OldPCB.registers[0..31] = CPU.registers[0..31]`
- `OldPCB.stackPointer = CPU.stackPointer`
- `OldPCB.state = READY` (أو WAITING)

2. تحديث Scheduler:

- إذا `state == READY`: أضف لـ Ready Queue
- إذا `state == WAITING`: أضف لـ Waiting Queue

3. اختيار Process جديد:

- Scheduler يختار من Ready Queue
- حسب الخوارزمية (FCFS, SJF, Priority, RR)

3. اختيار Process جديد:

- Scheduler يختار من Ready Queue
- حسب الخوارزمية (FCFS, SJF, Priority, RR)

4. استرجاع سياق Process الجديد:

- `CPU.programCounter = NewPCB.PC`
- `CPU.registers[0..31] = NewPCB.registers[0..31]`
- `CPU.stackPointer = NewPCB.stackPointer`
- `NewPCB.state = RUNNING`

5. تحديث Memory Management:

- تحديث Page Table
- Flush TLB (Translation Lookaside Buffer)
- تحديث Cache

مثال عملي مفصل

السيناريو: عمليتان A و B

الحالة الابتدائية:

- **Process A:** Running على CPU
  - PC = 1000
  - R1 = 5
  - R2 = 10
- **Process B:** Ready في Queue
  - PC = 2500
  - R1 = 100
  - R2 = 200

Timer Interrupt حصل!

الخطوة	الحدث	Process A	Process B	CPU
0	الحالة الابتدائية	Running (PC=1000)	Ready (PC=2500)	PC=1000, R1=5
1	Timer Interrupt	Running	Ready	توقف
2	حفظ سياق A	A.PCB.PC=1000, R1=5	Ready	-
3	تحديث حالة A	State=READY	Ready	-
4	إضافة A للـ Queue	في Ready Queue	Ready	-
5	Scheduler يختار B	في Queue	Selected	-
6	استرجاع سياق B	في Queue	Running	PC=2500, R1=100
7	تحديث حالة B	في Queue	State=RUNNING	PC=2500
8	استكمال التنفيذ	في Queue	يعمل ✅	B يشتغل

النتيجة:

- **Process A:** في Ready Queue، ينتظر دوره
- **Process B:** يعمل على CPU من حيث توقف (PC=2500)
- **Context Switch:** كلف حوالي 1000-2000 cycles

التكلفة والأداء (Overhead)

تحليل التكلفة:

العملية	الوقت (cycles)	الملاحظات
حفظ Registers	50	32 register × ~1-2 cycle
حفظ PCB (Memory Write)	100	كتابة في الذاكرة
Scheduler Execution	200	اختيار Process جديد
استرجاع Registers	50	قراءة من PCB
تحديث Memory Management	500	تغيير Page Table + Flush TLB
Cache Pollution	+1000	الآن فيه بيانات قديمة Cache
الإجمالي	**~2000**	**~1 microsecond**

⚠️ المشكلة:

Time Quantum	Useful Work	Overhead	% Overhead
100ms	99.999ms	0.001ms	✅ 0.001%
10ms	9.999ms	0.001ms	✅ 0.01%
1ms	0.999ms	0.001ms	✅ 0.1%
100µs	99µs	1µs	⚠️ 1%
10µs	9µs	1µs	❌ 10%

القاعدة الذهبية:

Time Quantum يجب يكون >> Context Switch Time

عادة: 10-100 milliseconds

🔢 القسم السابع: Program Counter (PC)

التعريف الأساسي

Program Counter (PC):

سجل خاص في CPU يحتوي على عنوان التعليمة التالية للتنفيذ

الأسماء البديلة:

- **Instruction Pointer (IP)** في معمارية Intel x86
- **Program Address Register** في بعض المعماريات

دورة Fetch-Execute

CPU Cycle (دورة المعالج):

1. Fetch (جلب):

- قراءة التعليمة من الذاكرة عند عنوان PC
- `instruction = Memory[PC]`

2. Decode (فك الترميز):

- تحليل التعليمة
- ما العملية؟ ما المعاملات؟

3. Execute (تنفيذ):

- تنفيذ العملية الفعلية

4. Update PC:

- تعليمة عادية: `PC = PC + 4` (في 32-bit)
- قفزة (Jump): `PC = target_address`
- فرع (Branch): `PC = condition ? target : PC + 4`

أنواع تحديث PC

نوع التعليمة	كيف يتحدث PC	مثال
تعليمة عادية	PC = PC + 4	`ADD R1, R2, R3`
قفزة غير مشروطة	PC = عنوان الهدف	`JMP 3000`
قفزة مشروطة	PC = شرط؟ هدف : PC+4	`BEQ R1, R2, 3000`
استدعاء دالة	حفظ PC، ثم PC = عنوان الدالة	`CALL function`
عودة من دالة	PC = العنوان المحفوظ	`RET`
System Call	PC = System Call Handler	`TRAP`

مثال 1: تعليمات متتالية

```text
Memory:
1000: ADD R1, R2, R3   // R1 = R2 + R3
1004: SUB R4, R1, R5   // R4 = R1 - R5
1008: MUL R6, R4, R7   // R6 = R4 * R7
1012: DIV R8, R6, R9   // R8 = R6 / R9
```

التنفيذ:

الخطوة	PC	التعليمة	الإجراء
1	1000	ADD	تنفذ → PC = 1004
2	1004	SUB	تنفذ → PC = 1008
3	1008	MUL	تنفذ → PC = 1012
4	1012	DIV	تنفذ → PC = 1016

مثال 2: قفزة (Jump)

```text
Memory:
2000: MOV R1, #10      // R1 = 10
2004: JMP 3000         // قفز إلى 3000
2008: ADD R1, R1, #1   // هذه لن تنفذ!
2012: SUB R1, R1, #1   // هذه أيضاً لن تنفذ!
...
3000: MUL R1, R1, #2   // R1 = R1 * 2
3004: ...
```

التنفيذ:

الخطوة	PC	التعليمة	الإجراء
1	2000	MOV	R1=10 → PC=2004
2	2004	JMP 3000	⭐ **PC=3000**
3	3000	MUL	R1=20 → PC=3004

مثال 3: System Call

```text
:User Code
:4000  MOV R0, #5        // syscall number
:4004  TRAP              // System Call!
:4008  ADD R1, R1, #1    // بعد System Call
```

التنفيذ:

الخطوة	Mode	PC	الإجراء
1	User	4000	MOV ينفذ → PC=4004
2	User	4004	TRAP ينفذ
3	**Switch**	4004	حفظ PC=4008 في PCB ⭐
4	Kernel	50000	PC=syscall_handler
5	Kernel	50000-50100	معالج System Call يعمل
6	**Switch**	-	استرجاع PC=4008 من PCB
7	User	4008	ADD ينفذ ✅

⭐ **نقطة مهمة:**

PC يحفظ في PCB حتى يعرف Process من وين يكمل بعد System Call!

PC في Context Switch

عند Context Switch:

Process A (Running):

- PC = 1500 (وصل لهذه التعليمة)

Context Switch يحصل:

1. **حفظ:** `A.PCB.savedPC = 1500` ⭐
2. A يدخل Ready Queue
3. Process B يستلم CPU
4. **استرجاع:** `CPU.PC = B.PCB.savedPC` (مثلاً 7200)

Process B (Running):

- PC = 7200 (يكمل من هنا)

لاحقاً، عند رجوع A:

- `CPU.PC = A.PCB.savedPC = 1500` ✅
- A يكمل **بالضبط** من حيث توقف!

🔀 القسم الثامن: Concurrency - التزامن

التعريف الأساسي

Concurrency (التزامن):

تنفيذ عدة Processes "في نفس الوقت" عن طريق مشاركة CPU

الفرق عن Parallelism:

الميزة	Concurrency	Parallelism
التعريف	مشاركة CPU بالتناوب	تنفيذ فعلي متزامن
CPUs	1 CPU أو أكثر	عدة CPUs
في نفس اللحظة	عملية واحدة فقط	عدة عمليات
الآلية	Context Switch سريع	تنفيذ حقيقي متوازي
مثال	Single-core processor	Multi-core processor

التشبيه:

- **Concurrency:** معلم واحد يدرس 3 طلاب بالتناوب (يعطي كل واحد دقيقتين)
- **Parallelism:** 3 معلمين يدرسون 3 طلاب في نفس الوقت

المشاكل الناتجة

1. Race Condition (حالة السباق):

التعريف:

عمليتان أو أكثر تصلان لـ Shared Data في نفس الوقت، والنتيجة تعتمد على ترتيب التنفيذ

2. Deadlock (المأزق):

التعريف:

عمليتان أو أكثر كل واحدة تنتظر الأخرى، والجميع محجوزين للأبد

3. Starvation (المجاعة):

التعريف:

عملية لا تحصل على CPU أبداً (أو نادراً جداً)

التعريف:

الجزء من الكود الذي يصل إلى Shared Data

القواعد الثلاث:

1. Mutual Exclusion (الاستبعاد المتبادل):

واحد فقط في Critical Section في أي لحظة

2. Progress (التقدم):

إذا ما فيه أحد في Critical Section، يجب السماح لأحد بالدخول

3. Bounded Waiting (انتظار محدود):

لا يجب Process ينتظر للأبد (تجنب Starvation)

📊 المقارنة الشاملة

جدول المفاهيم الثمانية

المفهوم	الوصف المختصر	متى يحدث	التأثير على CPU	Overhead
Interrupt	إشارة توقف CPU	جهاز I/O ينهي، Timer	توقف مؤقت + معالج	~100 cycles
User/Kernel Mode	مستويات الحماية	System Call، Interrupt	Mode Switch	~100 cycles
Process	برنامج قيد التنفيذ	fork(), exec()	استهلاك موارد	-
System Call	طلب خدمة من OS	open(), read(), fork()	User→Kernel	~250 cycles
Context Switch	تبديل العمليات	Interrupt، Timeout، I/O	حفظ/استرجاع كامل	~2000 cycles
Program Counter	عنوان التعليمة التالية	كل Instruction	PC++ أو Jump	0 (تلقائي)
Concurrency	تنفيذ متزامن	Multi-tasking	مشاركة CPU	Context Switch
Process States	حالة العملية	أحداث مختلفة	Scheduling	-

العلاقات بين المفاهيم

السيناريو الكامل: دورة حياة Process

المرحلة	الحالة	المفهوم المستخدم	التفصيل
1	NEW	Process	fork() System Call أنشأ العملية
2	READY	Process States	ينتظر في Queue
3	RUNNING	Context Switch	Scheduler اختاره، استرجع PC من PCB
4	RUNNING	Program Counter	CPU تنفذ تعليمات، PC يزيد
5	RUNNING	System Call	استدعى read(). User→Kernel Mode
6	WAITING	Process States	ينتظر I/O، لا يستهلك CPU
7	READY	Interrupt	I/O انتهى، Interrupt أشعر Scheduler
8	RUNNING	Context Switch	رجع لـ CPU، استرجع PC
9	READY	Interrupt	Timer Interrupt، Time Slice انتهى
10	RUNNING	Context Switch	رجع بعد عمليات أخرى
11	RUNNING	Concurrency	عمليات أخرى تعمل بالتوازي
12	TERMINATED	Process States	exit() System Call

مقارنة التكاليف (Overhead)

العملية	التكلفة (cycles)	التكرار	التأثير الإجمالي
Function Call	~15	عالي جداً (آلاف/ثانية)	منخفض ✅
Mode Switch	~100	متوسط (عشرات/ثانية)	متوسط ⚠️
System Call	~250	متوسط	متوسط ⚠️
Context Switch	~2000	متوسط (10-100/ثانية)	عالي ⚠️
Process Creation	~10,000+	منخفض (نادر)	عالي جداً ❌

الخلاصة:

- **Function Calls**: رخيصة → استخدمها بحرية ✅
- **System Calls**: مكلفة → استخدم بحكمة (Buffer البيانات)
- **Context Switches**: مكلفة → Time Quantum مناسب (10-100ms)
- **Process Creation**: مكلفة جداً → استخدم Threads بدلاً (Chapter 4)

🎓أسئلة امتحانات

اختيار من متعدد - MCQ

1.ما هو الفرق الرئيسي بين	Process و	Program ؟

- A) Program يستهلك Memory، Process لا
- B) Process هو Program قيد التنفيذ ✅
- C) لا يوجد فرق
- D) Process أسرع من Program

*الإجابة: B - Process = Program in Execution *

2.متى يحدث	Context Switch	؟

- A) عند Function Call عادي
- B) عند Interrupt أو انتهاء Time Slice ✅
- C) فقط عند إغلاق البرنامج
- D) كل مللي ثانية بالضبط

*الإجابة: B - Context Switch يحدث عند Interrupt، System Call أو انتهاء ، Time Slice *

3.ما هي الحالة التي يدخلها	Process	عند طلب	I/O ؟

- A) Ready
- B) New
- C) Waiting ✅
- D) Terminated

*الإجابة: C - عند طلب I/O (مثل read)، Process ينتقل من Running إلى Waiting *

4.ما هو دور Program Counter (PC) ؟

- A) عد عدد البرامج
- B) يحتوي على عنوان التعليمة التالية للتنفيذ ✅
- C) يحسب الوقت
- D) يخزن البيانات

*الإجابة: B - PC يحتوي على عنوان التعليمة التالية للتنفيذ *

5.أي من التالي يسبب Mode Switch من User إلى Kernel ؟

- A) إضافة رقمين
- B) System Call ✅
- C) حفظ متغير
- D) Function Call عادي

*الإجابة: B - System Call يسبب الانتقال من User Mode إلى Kernel Mode *

6.ما هو Interrupt ؟

- A) خطأ في البرنامج
- B) إشارة توقف CPU عن عملها الحالي ✅
- C) نوع من الفيروسات
- D) أمر لإنهاء البرنامج

*الإجابة: B - Interrupt إشارة من Hardware أو Software توقف CPU لتنفيذ معالج *

7.ما هي المشكلة الرئيسية في Concurrency ؟

- A) السرعة
- B) الذاكرة
- C) Race Condition والحاجة لـ Synchronization ✅
- D) الطاقة

*الإجابة: C - Concurrency يحتاج Synchronization لتجنب Race Conditions *

8.أي System Call ينشئ Process جديد؟

- A) read()
- B) write()
- C) fork() ✅
- D) open()

*الإجابة: C - fork() ينشئ نسخة طبق الأصل من Process الحالي *

9.ما الذي يُحفظ في Context Switch ؟

- A) Program Counter و Registers فقط
- B) Registers فقط
- C) Program Counter ،Registers ،Stack Pointer ،State ✅
- D) الذاكرة كاملة

*الإجابة: C - Context Switch يحفظ PC ،Registers ،Stack Pointer ،Stateومعلومات ، Memory *

10.في أي Mode يعمل نظام التشغيل؟

- A) User Mode
- B) Kernel Mode ✅
- C) Safe Mode
- D) Debug Mode

*الإجابة: B - نظام التشغيل يعمل في Kernel Mode مع وصول كامل للـ Hardware *

11.عند انتهاء I/O ، إلى أي حالة ينتقل Process ؟

- A) Running
- B) Ready ✅
- C) New
- D) Terminated

*الإجابة: B - من Waiting إلى Ready (ليس Running مباشرة!)